

Comments on “Everything is here: from do to html”

June 2026

This paper introduces two commands, `ishere` and `tohtml`, which together facilitate the production of HTML reports. The user writes a Stata do file logging the output and using `ishere` to insert headings, tables, figures, displayed results and bullet points, and then runs `tohtml` on the log to convert it to Markdown and then optionally to HTML.

The final result may include (1) just the headings, tables, figures, and the narrative with quoted results, my favorite option, or (2) that plus the Stata commands used to produce the results, or (3) that plus the Stata output in full. The examples in the paper start by showing everything, and then “clean” the report removing commands and console output.

The comparative advantage of this particular solution, compared to the official Stata commands `dyndoc` and `dyntext`, or the community-contributed commands `markdoc`, `markstat`, `webdoc` and others, is claimed to be its ease of use, with no steep learning curve. This is certainly true as long as one sticks to the items listed at the start of this review. It is also self-contained, requiring no external tools beyond Stata.

But that is not the whole story. The text blocks between the commands `ishere /*` and `ishere */` can be standard Markdown as understood by Stata’s `markdown` command. So one could use **bold**, *italics*, `code`, bulleted and numbered lists, in short, all of Markdown. In fact one can even use mathematical equations written in LaTeX and rendered by MathJax. But all of this is barely mentioned in the paper, with a brief remark near the end of page 5 that text blocks follow Markdown syntax, **so * is not a comment symbol but “a list marker”, and “mathematical equations can also be added”**. Of course using these features comes with a steep learning curve.

The comments below consider each of the two commands in turn, discuss the examples, and briefly touch on the ado and help files. I’ll follow essentially the same order as the paper. For ease of reference I have numbered all the comments.

The `ishere` Command

1. I was a bit confused by the statement that `ishere` has three modes, as I immediately wondered how do I know which mode it is in, or how do I change modes. Turns out these are more like what the Stata documentation calls syntax 1 or 2. The command `ishere` followed by `fig|figure`, `tab|table` or `display`, inserts something in the log (mode 2), and if followed by anything else –including nothing at all– it is just echoed to the log file (mode 1) as a marker for `tohtml`. Perhaps adding “depending on the syntax used” the first time modes are mentioned would help.

2. The discussion of mode 1 (page 3) shows several uses of `ishere` as placeholder, with `**` as an alternative. I liked the introduction of `ishere` with a reference to the traditional “include Table 1 here” as motivation, and wonder if we really need this alternative, which can only be used in mode 1.

3. The first item in the list has `ishere```` and `**```` as code boundaries, which are required before Stata code. One concern here is that ````` is a well-established Markdown convention used to mark code blocks, and it is always used in pairs, one before and one after the block, not just before. Also `ishere` by itself appears to have the same effect and is simpler, so I'd just lose the backticks.
4. A second issue is that there should be no need to mark both Stata and Markdown blocks. The code in `tohtml` should be able to figure out when it is dealing with Markdown and when with Stata commands and output. All other tools use either a do file with special markings for Markdown blocks (for example `markdoc`, with `/**` and `**/`), or a Markdown file with special markings for Stata commands (for example `dyndoc`, with dynamic tags), and that's enough to distinguish both. This would reduce the risk of errors.
5. The discussion of mode 2 (page 4) shows the HTML inserted by several `ishere` commands. I am not sure how helpful this will be to the target audience, but perhaps clarify that "the examples below show various `ishere` commands and the HTML they generate".
6. The third mode happens when `ishere` is not used as a command at all, but appears in the narrative to quote a result. The syntax here is a bit cumbersome. Looking at the example on page 5, to report an R-squared you have to store `e(r2)` in a local macro `r2`, add `ishere display %5.3f `r2'` to display the value, and finally add a text block

```
ishere /*
* The model R-squared is {ishere display %5.3f `r2'}
ishere */
```

an instruction for `tohtml` to pick up the value and insert it in the narrative. The issue here is that one must type *exactly* the same thing twice, otherwise the value will not be picked up. Not clear to me how to make this simpler.

The `tohtml` Command

7. This command does all the heavy lifting. It also has two modes (or syntaxes), but it is immediately obvious that mode 1 has as first argument a log file and mode 2 has a folder name. In fact one or more directories as noted on page 7.
8. In mode 1 the log will be translated to Markdown and optionally HTML. The Markdown file is named by appending `_clean.md` to the log name, or the user may specify a name with the option `cleanmd(filename)`. Not sure why the emphasis on "clean", this is just a translation of the log to Markdown.
9. HTML is generated when the `html(string)` option is used. The required argument is a filename and the extension `.html` is nicely added if not present, although this is not mentioned. Wouldn't it be nice if one could specify just `html` and have the filename default to the name of the log, say `logname.html`? If the user wants a different name the option would then read `html(filename)`. Just a thought.
10. When `html` is used you have the option of specifying a CSS file for styling the output. This is either `css(githubstyle)` or `css(filename)`, so presumably no CSS is the default.

Why not make the `githubstyle` the default? This is used in all of the examples.

11. The `clean` option, my personal favorite, removes all commands and Stata output, leaving just the headings, narrative, tables and figures. By the way this is not “minimal cleaning mode”, if anything it is minimal *output mode*.

12. Then we come to the `cleancode(dofilename)` option, which removes only the Stata output, leaving all the above plus the commands. I was surprised that this requires the original do file, as Stata echoes all commands in the log file. The concern here is that results are unpredictable if the do file mentioned in the option differs from the file actually used to generate the log. Saying “the file must contain `ishere` markers” is not enough; it has to be the same.

13. I understand `cleancode` as an alternative to `clean`, but there is no check that only one of these options is used. If both are specified `cleancode` is ignored. I am also not sure why this is called “clean” code, as it hasn’t been tidied or removed.

14. The options to hide code and output, or just output, are global; in contrast with other tools that allow finer control at the command level. This is clearly one of the tradeoffs in the interest of simplicity. Note, however, that one can still suppress output using Stata’s `quietly`.

15. I don’t understand the `rpath` and `wpath` options, which are supposed to help generate portable reports. Perhaps a short example would help.

Comparisons

16. I found the comparison with alternative solutions starting on page 8 useful, perhaps akin to a literature review, but a bit repetitive and occasionally contentious.

17. It notes that `stmd` requires you to maintain a separate document, but here we must maintain a do file. The only advantage would be that a do file is a more familiar working environment.

18. Similarly, it is stated that `markdoc` requires writing Markdown within special comment delimiters while the present system is simpler; but this is only true if you stick to headings, figures and tables. Everything else also requires writing Markdown within special comment delimiters.

19. In the same vein it notes that `markstat` mixes a narrative and commands on the same file, but so do `ishere/tohtml` in the do file, so I don’t see the clear separation of concerns.

20. The comparison with `texdoc/webdoc/sttex` is devoted mostly to `sttex`, which is really a system for inserting Stata results in a LaTeX file, not an alternative to the current offering at all. The same is true of `texdoc`. I think `webdoc` is more relevant.

Examples

21. Example 1 on page 11 basically produces an HTML log file with a heading. It’s a bit like `log2html` plus the heading minus the highlighting of input and output. I think this example

and all others should start with `capture log close`, because if anything goes wrong the log will be left open. Also one *must* remember to close the log file before calling `tohtml`.

22. There is a problem with the generated Markdown in all the examples processing a log. The first five lines of a Stata text log file have the form

```
-----  
    name: <unnamed>  
    log: filename.log  
    log type: text  
opened on: 1 June 2026, 14:12:32
```

These lines are written unchanged to the “clean” Markdown file. When that is run through `markdown`, the dashes become a horizontal rule, the next two lines are treated as code because they are indented, and the final two lines are treated as a single paragraph. I believe this block needs code fences before and after for correct rendering.

23. The generated Markdown in examples 1 and 2 have an odd number of code fences, which is an error. Fortunately Stata’s `markdown` is forgiving and closes the last open block.

24. Interestingly, example 1 marked the start of a code block with `ishere` with no arguments, whereas example 2 used `**```. I changed all of these to `ishere` and it worked fine. Same is true in other examples. I think `ishere` is better, but as noted earlier it would be best if it was not required.

25. Example 3 has a different problem. This is an illustration of quoting a result in the narrative. Here the `log close` comes up after the narrative, without an `ishere` to introduce it. As a result, the text is interpreted as Markdown and everything up to the date and time the log closed is rendered as a level 2 heading. This occurs because the final line of dashes is interpreted as underlining to indicate a heading. My point here is that if the developers can forget inserting a required `ishere`, so will many users.

26. Example 4 didn’t run at first because `outreg2e` was not found. The paragraph *after* the example notes that the examples require `outreg2e` and `logoute`, which are extensions by the authors of the popular commands `outreg2` and `logout` by Roy Wada. After installing these the example runs fine, except that the rendering problem caused by a missing `ishere` noted above also occurs here.

27. The text notes that one can also use Stata’s built-in `table` command (and presumably `dtable` and `etable`), but when I tried this I noticed that the table lacked the expected style. Turns out the `collect export` commands may generate CSS in addition to HTML, but `ishere/tohtml` ignore that. The CSS file has the same name as the table, unless the user has specified a different `cssfile`. I think `ishere` would have to insert a stylesheet link as the last line of the HTML table, after checking that it is not there already.

28. Example 5 shows off clean mode but unfortunately didn’t work for me. First `tohtml` objected to the `text` option, which is not allowed, so I removed it. Then the frame under “Main Results” was empty. Here the story gets long, so bear with me.

- a. There is an inconsistency in the call to `outreg2e` because it reads `using`

"main_results.html", yet requests a Markdown table using the `md` option. As it happens `outreg2e` wisely ignores the extension and produces `main_results.md`, which is what `ishere` then uses. Should make this consistent.

- b. Then `ishere` generates the code `<iframe main_results.md></iframe>` (see also page 4), which makes it unchanged to the HTML file, but is not valid HTML. It looks to me as if this is just a marker for `tohtml`, which should then remove the `iframe` and insert the Markdown in the document, but that doesn't happen. This is a bug that needs to be fixed.
- c. In the meantime my solution was to change the call to `outreg2e` to specify the `html` option and change the `ishere` call to use `main_results.html`, which worked fine and produced a nice looking report.

29. The Markdown file here is `example5.clean.md` with a `.` before `clean`, not the expected `_` used in examples 1-4 and 6. This suggests the code is not properly factored, with a single place where the output Markdown file is named.

30. All of the examples that use a text block use only bullet points, which means all lines start with `*`. A reader might think that is required when it is not.

31. Section 3.7 (page 15) has two unnumbered examples that generate figures and tables and use `tohtml` in directory mode to gather them. This failed because command `fs` was not unrecognized. I thought this could be Nick Cox's `fs`, installed it, and it all worked fine. It is not the example that calls `fs`, but rather `tohtml` in mode 2.

32. I wondered if the tables and figures had to be in separate folders, as done in these examples with the folders `output/figures` and `output/tables`, but moving them all to `output` worked too, as long as the names of the files start with `figure` and `table`.

Implementation

33. The `ishere` command is the simplest. I noticed that in placeholder mode the code does not check that the argument is one of the options listed at the bottom of page 3 and top of page 4. If you type anything else it goes into the log, and `tohtml` will render it as code or Markdown depending on the context. Also the command must fit in one line; continuations don't work.

34. In insertion mode the arguments are lowercased by the code, so you can say `ishere` `FIGURE` using `scatter.png` and it will work. Why depart from standard Stata practice? Also `ishere` `DISPLAY` passes the check at the top of the code, but doesn't display anything because later code requires lowercase.

35. The `tohtml` ado file starts with `capture program drop tohtml`. This should be dropped in the final version.

36. The program then checks for the existence of `pathutil` and if not found installs it from SSC. In my opinion a command should not install other commands without an action from the user. Just note the dependency, and perhaps provide a link to install.

37. Turns out `tohtml` also requires `mm_outsheet` from Ben Jann's `moremata` library. This worked for me because I happen to have it installed, but that will not be the case for all users. Treat as suggested above. Or, given that all you need is to write a string vector to a file, write a small function to do that and save the library install.
38. One more dependency already noted is Nick Cox's `fs` command. This should also be checked here in a "dependencies" section with a link to install.
39. When generating Markdown, `tohtml` inserts a space at the start of every line. This conflicts with Markdown code that must start in column 1, such as headings (both `setext` and `atx`). One can always enter headings with `ishere # Heading`, but it would be nice to be able to enter both a heading and a paragraph in the same text block.
40. The description in Section 4 refers to a "translated" log file, but the workflow specifies using a text file. Maybe this is because one might use a SMCL log, which must then be translated. Better to be explicit.
41. Item 4 in the parsing strategy is "code block inference", which in my opinion is not really implemented if to detect Stata code the user has to flag it with `ishere`.
42. The discussion of CSS and MathJax integration is puzzling. Why make the use of MathJax depend on using the `githubstyle` CSS? What if a user wants to use both MathJax and a custom CSS file? I feel that `mathjax` should be a separate option. Also, MathJax is a large javascript file, and there is no need to add a link if it is not used. Currently it is linked even if there are no equations in the file, as in all the examples.
43. The CSS default file is included in the code. I would suggest using an actual file, say `tohtml.css`, and designating it as a resource file to be installed in the same location as `tohtml` itself (an `F` file in a package). The method used to grow the `css` vector one line at a time is at least 20 times slower than starting with `css = J(149,1,"")` and then assigning each of the 149 lines. Not that anyone would notice because Mata is very fast in either case. A separate file also makes maintenance easier.
44. A final remark on implementation concerns self-contained files. Solutions that generate `docx` or `pdf` files have the advantage that they include all the tables and figures in the document. When one generates `html` files all they have are links to the tables and figures, as well as the CSS file(s). If you want to give a colleague a copy of your report you have to pass along all of these files.
- One solution is to create an archive or `zip` file with everything. This could be an option in `tohtml`, which could leverage Stata's `zipfile` to create the archive. Obtaining a list of tables and figures would be easy as they were inserted by `ishere`.
 - Another solution is to have an option to include the figures in the document in base 64 format, which is easy now that Mata has a `base64encode` function. For tables I would recommend including them directly in the document, together with their CSS if any. Both tasks are best handled by `tohtml` rather than `ishere`, as that avoids cluttering the log and the intermediate Markdown files.

The Help Files

45. Both commands have fairly good help files. However, they both include a “complete workflow example” that uses a SMCL log, in contrast with the paper, that uses text logs (see Section 2.4). Turns out the SMCL log must then be converted to text format using Stata’s `translate` command. Forgetting this step before running `tohtml` results in errors. Why not have `tohtml` convert the log automatically when necessary?

46. The `ishere` help file has many examples showing how to display a result. In all cases the result is first copied into a local macro. Why? And more interestingly, why doesn’t it work if one wants to display `e(r2)` without first storing it in a macro. Of all the examples, the only one that worked for me without a macro was displaying the number of observations `_N`. At the end of this file the recommended workflow again uses a SMCL file and translation to text.

47. The `tohtml` help file has a lot of examples. All of them refer to the input log file as `analysis.md`. This is confusing as `.md` is the standard extension for Markdown, and Stata logs are not in Markdown, they are either text with a `.log` extension or SMCL with a `.smcl` extension. Why depart from standard usage?

48. In the Remarks section, under dependencies, it says that generating HTML “requires Stata’s `markdown` command. Install via `. scc instal moremata`.” Well, `markdown` is part of Stata since version 15, it doesn’t have to be installed. As noted earlier Ben Jann’s `moremata` is required, and the same is true for Daniel Klein’s `pathutil` and Nick Cox’s `fs`.